01 AI项目为什么需要本体？–本体论在 AI 工程中的实践系列

陈乔 通往AI之桥 2026年10月25日 16:03 北京

从哲学概念，到可落地的工程语义层

通往AI之桥｜本体论在 AI 工程中的实践系列 01

当 AI 从“回答问题”走向“调用工具、推进流程、影响业务系统”，一个老问题重新变得重要：AI 对企业业务的理解，究竟依据什么？

01 一个看似抽象的问题，正在变成工程问题

今天，我们给大模型接入知识库、数据库和各种工具，似乎就可以让它“懂业务”。但在实际项目中，很多问题并不是模型不知道某段文字，而是不同系统对同一个业务概念理解不一致。

CRM 里的“客户”，可能是联系人；财务系统里的“客户”，可能是签约主体；订单系统里的“客户”，又可能是下单账号。一个系统把订单状态写成“完成”，另一个系统却要等签收和对账都结束才算“完成”。如果这些概念没有统一定义，AI 就可能找到正确的资料，却得出错误的业务判断。

当 AI 只负责生成一段文字，语义偏差也许只是一个回答不准确；当 Agent 开始查询数据、生成报价、审批工单、调整库存或执行部署，语义偏差就可能变成真实的业务风险。

AI 工程的难点，正从“能不能生成答案”转向“能不能在真实业务世界中，基于一致的含义可靠地行动”。**

02 本体从哪里来？从哲学问题走向知识工程

Ontology 通常译为“本体论”，在现代哲学中，它关心“存在是什么”“世界由哪些基本事物构成、它们如何关联”等问题。相关思考可以追溯到古希腊哲学，尤其是亚里士多德对“存在之为存在”的讨论。

“Ontology”这个术语则是在 17 世纪早期进入哲学词汇的，常被追溯到 Jacob Lorhard 1606 年的著作，随后出现在 Rudolph Goclenius 的哲学词典中，并在 Christian Wolff 等哲学家的著作中进一步发展。换句话说，名字并不新，问题也不是大模型时代才出现的。

到了人工智能和知识工程领域，Ontology 被赋予了更工程化的含义。Tom Gruber 在 1993 年的论文中，把本体描述为对某个共享领域中表示词汇的规范，涉及类、关系、函数等概念。后来广泛使用的一种表述是：本体是“对共享概念化的明确、形式化规

范”。W3C 的 OWL 标准则进一步支持以机器可处理的方式描述对象、类别及其关系，并进行一致性检查和逻辑推断。[11][2]

如果把哲学中的问题改写成工程语言，本体要回答的是：我们研究的这个业务领域里，究竟有哪些“东西”？每一种东西是什么意思？它们之间有哪些关系？哪些规则必须成立？这些定义如何被人、软件和 AI 共同使用？

03 从概念到工程制品：本体究竟包含什么？

先看一个简单的电商例子。不要从数据库表开始，而是从业务世界开始：

– 实体类型：客户、订单、商品、付款、物流单。

– 属性：订单金额、下单时间、订单状态、商品 SKU。

– 关系：客户创建订单；订单包含商品；付款对应订单；物流单履约订单。

– 规则与约束：已取消订单不能进入正常发货流程；退款必须关联一笔可识别的支付记录；订单状态的变化需要满足明确的前置条件。

– 动作：创建订单、批准退款、分配仓库、确认发货。动作还要定义执行权限、必需输入和允许产生的状态变化。

在真实工程中，还应进一步描述每个概念的数据来源、唯一标识、口径、时间语义、权限、版本和责任人。否则，本体可能只是漂亮的图，而不是可以落地使用的工程契约。

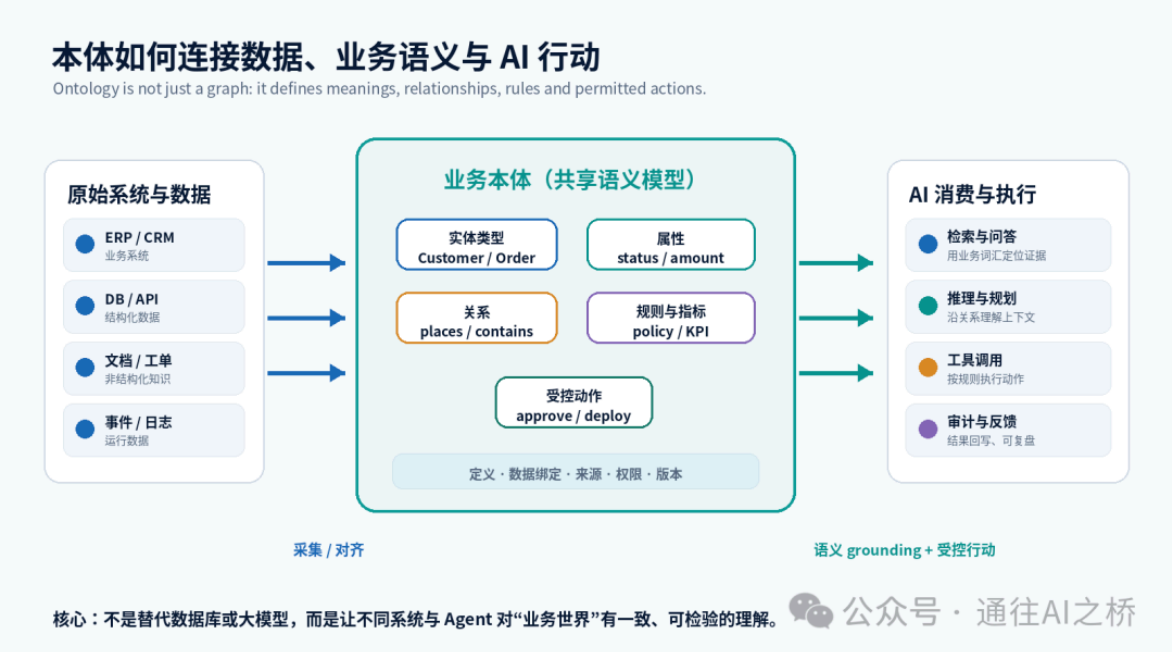


图 1 | 本体把数据源、业务概念、关系、规则与受控动作连接起来。

04 本体不等于数据库 Schema，也不等于知识图谱或 RAG

这些概念经常一起出现，但它们解决的问题不同。

概念	核心回答	与本体的关系
数据库 Schema	数据在某个系统里如何存储?	说明表、字段、类型和约束；本体关注跨系统共享的业务含义。
分类体系 / Taxonomy	事物如何分门别类?	分类层级是本体的一部分，但本体还可以表达多种关系、属性、规则与约束。
知识图谱	实体和关系的数据如何组织、查询?	图谱可按本体定义来实例化和校验；但并非每个知识图谱都包含严格、完整的本体。
RAG / 向量检索	哪些材料与当前问题相关?	RAG 负责检索证据；本体提供概念含义、关系和业务约束，两者可以互补。
大语言模型	如何理解语言、归纳和生成?	模型负责泛化与生成；本体给出共享语义和显式边界，并不替代模型。

公众号 · 通往AI之桥

一个实用的理解是：RAG 帮 AI 找到相关内容；本体帮助 AI 理解这些内容里的“客户”“订单”“版本”“依赖”到底指什么，以及它们之间允许成立什么关系。若问题涉及复杂关系，知识图谱或 GraphRAG 可以把这些关系用于检索和推理。

05 主流厂商怎样定位本体？几条值得关注的路线

各家厂商并没有统一使用 Ontology 这个标签。有些将本体做成产品的一等概念；另一些采用知识图谱、语义模型或 GraphRAG 等相邻技术。把它们放在同一张地图上看，反而更容易理解这个方向。

Palantir：从“数据层”走向企业的操作层

Palantir Foundry 将 Ontology 定位为组织的 operational layer（操作层），把数据集、模型等数字资产映射到现实业务对象。它用对象类型、属性、链接来描述业务世界，又用 Actions、Functions 和动态权限来表达怎样改变现实业务状态。官方强调，本体不只是目录或 Schema，而是连接数据、逻辑与行动的基础，并能让 AI 与人协同完成受治理的工作流。[3]

这是目前最鲜明的“操作型本体”路线：本体里既有业务名词，也有业务动作。它的价值不止在于让 AI 回答得更准确，更在于把决策连接到真实系统，并保留控制与反馈。

Microsoft：Fabric IQ 把共享业务语义交给数据与 Agent 使用

Microsoft Fabric IQ 的 Ontology 功能目前仍标注为预览版。微软将其定义为可由机器理解的共享业务表示，描述实体类型、属性、关系、规则、指标，并绑定到 OneLake 中的企业数据。它可以从既有 Power BI 语义模型生成或对齐，使业务术语和 KPI 在报告、应用和 Agent 之间保持一致；Agent 可在这些定义的上下文中提问、分析和提出受治理的行动。[4]

这条路线把本体放在企业数据和智能体体验之间，强调“业务上下文层”。它也提醒我们，业务语义应尽可能复用已验证的指标定义，而不应让每个 AI 应用各自解释指标。

华为云：把本体建模作为知识图谱的第一步

华为云知识图谱服务明确把本体设计列为构建流程的第一步，支持领域专家参与建模，定义实体类别、关系和属性，再进行信息抽取、知识映射、知识融合与知识服务。平台也提到利用模式学习从文档中发现典型结构，辅助人工提高本体设计效率。[5]

它体现的是“知识工程 + 本体 + 图谱”的路线：先确定领域语义结构，再让数据进入结构化知识体系。对于医疗、制造、运维等专业领域，这种方式尤其有参考价值。

百度、腾讯、AWS、Oracle、Google：与本体相邻的知识图谱和语义增强路线

百度智能云知识图谱提供面向行业的构建与应用流程，包括数据生产、知识抽取、融合、搜索、问答和推理。[6] 腾讯云 GraphRAG 将实体、关系和属性抽取与语义检索结合，目标是支持跨文档关联和多跳推理。[7] AWS Bedrock Knowledge Bases 可以与 Neptune 结合构建 GraphRAG；Oracle AI Database 26ai 也展示了把向量检索与属性图结合的架构模式；Google Cloud 则提供 Enterprise Knowledge Graph 等知识整合能力。[8][9][10]

这些方案并不都把“企业本体”作为同等明确的产品中心，不能直接说它们就是本体平台。但它们说明了一个共同趋势：单纯相似度检索并不总能满足复杂业务问题，实体、关系、来源与语义上下文正在成为企业 AI 工程的重要组成部分。

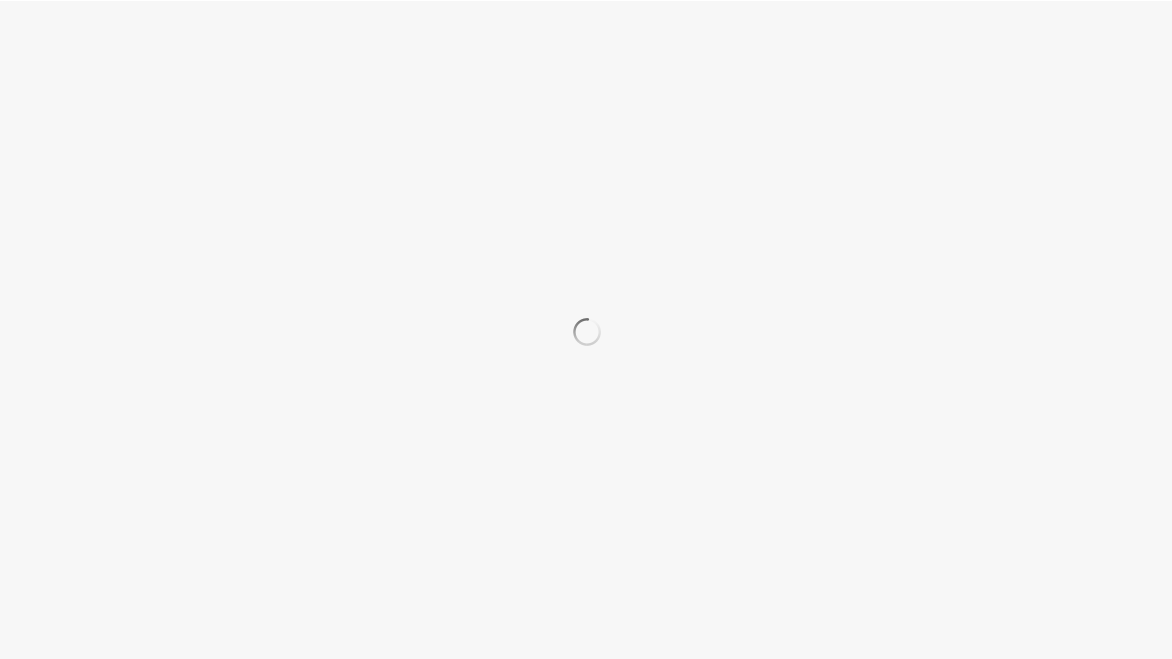


图 2 | 从分散的数据源，经过共享语义层，最终连接到多种 AI 能力与受控行动。

06 为什么 AI 项目需要本体？五个工程理由

理由一：减少不同 Agent、系统之间的“语义漂移”

多 Agent 系统常把任务拆给不同角色：需求分析 Agent、数据分析 Agent、开发 Agent、运维 Agent。若每个 Agent 都通过提示词临时理解“客户”“风险等级”“已交付”

“高优先级”的含义，即使使用同一个模型，也可能形成不同解释。本体提供共享概念、关系和定义，使多个 Agent 在同一语义基础上协作。

理由二：让 RAG 从“找到一段话”迈向“理解关系”

问题的答案有时不在一篇文档里，而在多段事实的连接上：某个组件由哪个版本引入、依赖什么库、影响哪些服务、有哪些漏洞、当前哪些客户部署了它。向量检索可以找到相关片段；实体和关系模型能进一步组织连接路径，帮助系统进行多跳检索与核验。

理由三：为 Agent 的行动提供可检查的边界

当 Agent 从建议走向执行，风险就不再只是文本错误。本体可以将对象、动作及其前置条件描述清楚，例如“只有完成测试并通过安全门禁的构建，才可以进入发布审批”。真正的执行仍必须由工作流、权限系统、策略引擎和审批机制保障；本体提供可共享的语义基础，但不能单独替代这些控制。

理由四：让结果更容易评估、解释和审计

如果系统输出“建议升级组件”，工程团队需要知道它指的是哪个组件、哪个版本、哪些服务受到影响、依据哪些依赖关系和漏洞证据。显式的对象、关系、来源和规则，使答案可以被检查，并能把“正确与否”的评估从主观感受转向业务约束和测试用例。

理由五：把一次性上下文变成可复用的企业资产

没有共享语义层时，每一个 Agent 项目都可能重新整理实体名、指标定义、关联规则和数据映射。有了本体，概念和关系可在知识问答、分析、流程自动化、软件开发、运维治理之间复用。长期看，这种可复用资产可能比某个 Prompt 或单次 Agent 编排更有积累价值。

本体最值得投入的地方，不是让一个 Demo 看起来更聪明，而是让多个 AI 应用在多次迭代中持续使用同一套经过治理的业务含义。

07 把本体放进 AI 工程：以“软件工厂”为例

如果把本文的思路放回软件工厂，本体可以描述的不只是客户或订单，也包括软件生产本身的对象：Requirement（需求）、Product（产品）、Component（组件）、Version（版本）、Build（构建）、TestSuite（测试集）、Finding（安全/质量发现）、Release（发布）、Environment（环境）、Deployment（部署）和Incident（事件）。

关系可能包括：需求“由”版本实现；组件“依赖于”另一组件；构建“由”某个提交产生；构建“通过”哪些测试；发布“包含”哪些制品；某版本“部署到”哪些客户环境；漏洞“影响”哪些版本。规则则可以明确：未通过强制测试的构建不可发布；高危漏洞未豁免不可进入生产；客户化版本必须能追溯到基础产品版本和差异配置。

这样，需求 Agent、开发 Agent、安全 Agent、发布 Agent 和运维 Agent 不必各自发明一套概念。它们围绕同一组对象、关系和规则协作：AI 可以帮助执行任务，但哪个版

本是什么、哪些依赖真实存在、什么条件允许发布，不能只由模型临场猜测。

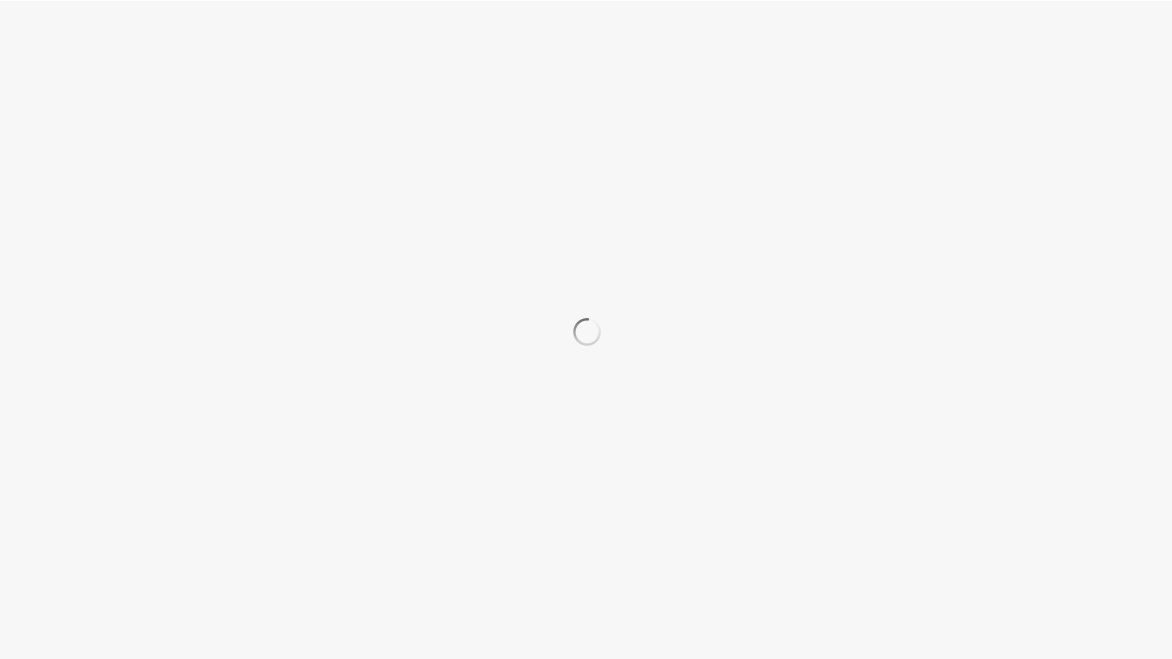


图 3｜电商示意图展示实体与关系；在软件工厂中，同样可以把需求、组件、版本、测试和部署组织成共享的业务世界模型。

08 不是所有项目都需要“重型本体”

本体不是万能药，也不是每个 AI 项目的必选项。如果只是单份文档的摘要、简单 FAQ，轻量元数据加普通 RAG 可能已经足够。如果业务概念变化极快、仅由一个小应用使用，建立复杂的形式化模型反而增加维护负担。

以下情况更值得认真考虑本体：多个系统对同一概念使用不同口径；多个 Agent 需要共享业务上下文；问题依赖跨系统关系和多跳推理；AI 将调用会影响业务状态的工具；或企业需要可审计的规则、指标与决策。

实践上也不建议一开始就试图建模整个企业。可以从一个高价值业务场景出发，先定义 10–20 个核心实体、关键关系、术语口径和少量强约束；绑定真实数据，用代表性问题验证；再将本体接入检索、Agent 和工作流，观察语义错误、人工返工和任务成功率是否改善。

09 本体的真正价值：为 AI 建立一份“业务语义契约”

回到开头的问题：为什么 AI 项目需要本体？不是因为本体比大模型更聪明，也不是因为有了本体，幻觉就会自动消失。

本体的价值在于把原本散落在人脑、表结构、需求文档、代码和流程里的业务含义，整理成可共享、可复用、可验证的表达。大模型擅长理解模糊表达、进行归纳并提出方案；本体擅长稳定概念、显式表达关系和约束。两者配合，才更有机会让 AI 从“能说”走向“能在规则内做事”。

我更愿意把本体理解为 AI 工程中的一份“业务语义契约”：人、系统和 Agent 可以围绕它沟通；数据可以映射到它；规则可以校验它；工具可以基于它执行动作；生产结果又可以反过来改进它。

当 AI 开始参与真实世界的决策与执行，我们需要的不只是更大的模型，还需要一套所有参与者共同理解的世界模型。**

系列预告

这是“通往AI之桥 | 本体论在 AI 工程中的实践”系列第一篇。后续我计划继续讨论：

– 第二篇：本体、知识图谱、语义层与 RAG，到底是什么关系？

– 第三篇：怎样从一个真实业务场景开始设计本体？

– 第四篇：本体怎样让 AI Agent 的工具调用更可靠、更可审计？

– 第五篇：如何把本体用于 AI 软件工厂、组件库与软件供应链？

也欢迎留言聊聊：在你的 AI 项目里，最难统一的业务概念是什么？是客户、订单、指标口径，还是系统之间的依赖和权限？

参考资料与延伸阅读

说明：以下资料用于核对历史定义、标准和厂商产品定位。厂商介绍反映公开产品表述，不等同于独立第三方效果评测。

– [1. Stanford Encyclopedia of Philosophy — Ontology and Information Systems](https://plato.stanford.edu/entries/ontology-is/)

– [2. W3C — OWL 2 Web Ontology Language Primer](https://www.w3.org/TR/owl-primer/)

– [3. Palantir Foundry — Ontology Overview](https://www.palantir.com/docs/foundry/ontology)

– [4. Microsoft Learn — What is ontology in Fabric IQ? (Preview)](https://learn.microsoft.com/zh-cn/fabric/iq/ontology/overview)

– [5. Huawei Cloud — Knowledge Graph, ontology design](https://www.huaweicloud.com/product/nlpkg.html)

– [6. Baidu AI Cloud — Knowledge Graph construction and application](https://ai.baidu.com/solution/kgaas)

– [7. Tencent Cloud — Knowledge Graph GraphRAG](https://www.tencentcloud.com/zh/document/product/1254/78733)

– [8. AWS Machine Learning Blog — Build GraphRAG applications using Amazon Bedrock Knowledge Bases](https://aws.amazon.com/blogs/machine-learning/build-graphrag-applications-using-amazon-bedrock-knowledge-bases/)

– [9. Oracle Developers — GraphRAG with Oracle AI Database 26ai](https://blogs.oracle.com/developers/graphrag-with-oracle-ai-database-26ai-knowledge-graphs-for-enterprise-ai-systems)

– [10. Google Cloud — Enterprise Knowledge Graph overview](https://docs.cloud.google.com/enterprise-knowledge-graph/docs/overview)

– [11. Tom Gruber — A Translation Approach to Portable Ontology Specifications (1993)](https://tomgruber.org/writing/ontolingua-kaj-1993/)

本体论在AI工程中的实践 · 目录 ≡

下一篇 · 02 本体、知识图谱、语义层与 RAG，到底是什么关系？ ›

作者提示: 内容由AI生成